Піктограма додавання Додати обкладинку

Phase 1: Task Tracker

Technical Requirements Specification

Version: 1.0

Date: 12.04.2026

Author: Valeri, EU Team Lead

Status: Draft

Audience: Development team (Anton, Maks)

Estimated scope: 6–8 weeks

---

1. Overview

Phase 1 delivers the core Task Tracker functionality: **a three-level entity hierarchy (Story → Task → Subtask)**, full CRUD operations, status and progress automation, commenting system, change log, and two primary views — Backlog (table) and Kanban board.

This module integrates with the existing QRT Digital Platform infrastructure (PostgreSQL, Go backend, React frontend, Redis caching) and connects to the People/HR module for assignee resolution.

---

2. Entity Hierarchy

The Task Tracker uses a strict three-level hierarchy. Each level serves a distinct purpose:

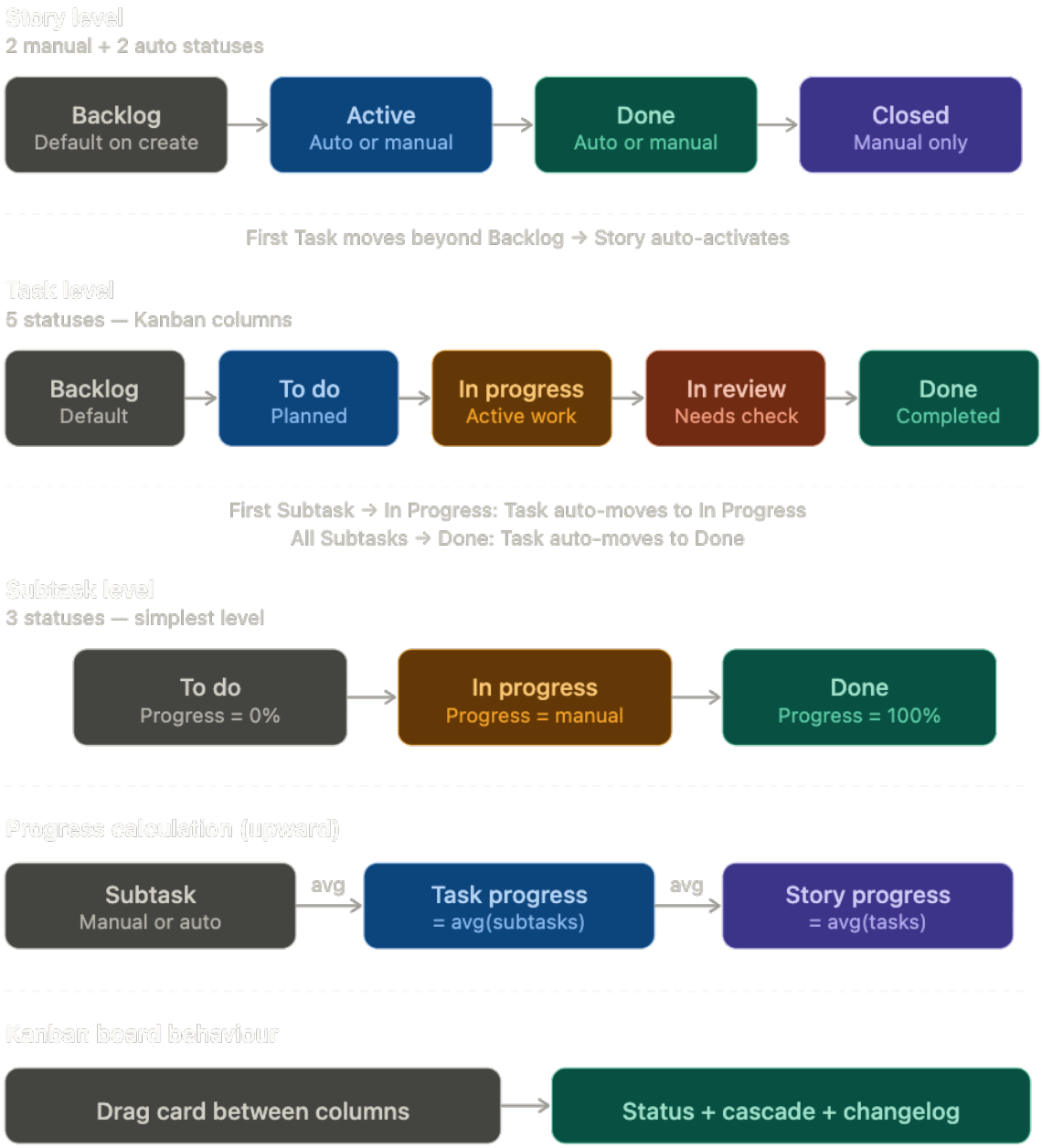
Story — a high-level unit of work representing a business objective, feature, or initiative. Stories group related tasks and provide the summary view for Team Leads and C-Level.

Task — a concrete unit of work within a Story, assigned to an individual. Tasks have start/end dates and represent trackable deliverables.

Subtask — an atomic work item within a Task. Subtasks break a Task into granular steps and are the lowest level at which progress is tracked.

Hierarchy rules:

- A Story contains 0..N Tasks
- A Task contains 0..N Subtasks
- A Subtask cannot contain children
- Tasks cannot exist without a parent Story
- Subtasks cannot exist without a parent Task
- Deleting a Story cascades to all child Tasks and Subtasks (with confirmation)
- Deleting a Task cascades to all child Subtasks (with confirmation)



Interface mocup

Stories отображаются как swim lane headers — цветные полоски с названием стори, процентом прогресса и мини-прогрессбаром. Они повторяются в каждой колонке, где есть их Tasks. Два примера: «Platform v2.0» (фиолетовый) и «Portal redesign» (зелёный).

Tasks — это карточки внутри колонок. Каждая карточка показывает: название, priority badge (Critical/High/Medium/Low с цветовым кодированием), аватар assignee (инициалы), счётчик subtasks (например «3/5»), и deadline (с красной подсветкой если overdue).

Subtasks — раскрываемый чеклист внутри карточки. На примере карточки «Command module — task CRUD» (выделена синей рамкой) видно 5 subtasks: 3 завершены (зелёная галочка, зачёркнутый текст), 2 ещё в работе. Это inline-расширение по клику на карточку.

Ключевые UX-решения в мокапе: карточки в «Done» слегка затемнены (opacity), overdue дедлайн выделен красным текстом, фильтры вверху позволяют переключаться между Stories и фильтровать по assignee/priority.

---

3. Data Models

3.1 Story

TypeScript

```typescript
interface Story {
  id: UUID;
  title: string;                    // max 255 chars
  description: string | null;       // rich text (Markdown)
  status: StoryStatus;
  progress: number;                 // 0–100, auto-calculated
  priority: Priority;
  project_id: UUID | null;          // FK → Projects module (optional)
  tags: string[];                   // custom labels for filtering
  created_by: UUID;                 // FK → Users
  assigned_lead: UUID | null;       // FK → People/HR (responsible lead)
  start_date: Date | null;
  deadline: Date | null;
  created_at: Timestamp;
  updated_at: Timestamp;
}

```

3.2 Task

TypeScript

```typescript
interface Task {
  id: UUID;
  story_id: UUID;                   // FK → Story (required)
  title: string;                    // max 255 chars
  description: string | null;       // rich text (Markdown)
  status: TaskStatus;
  progress: number;                 // 0–100, auto-calculated from Subtasks
  priority: Priority;
  assignee_id: UUID | null;         // FK → People/HR
  start_date: Date | null;
  deadline: Date | null;
  estimated_hours: number | null;
  sort_order: number;               // position within parent Story
  created_by: UUID;
  created_at: Timestamp;
  updated_at: Timestamp;
}

```

3.3 Subtask

TypeScript

```typescript
interface Subtask {
  id: UUID;
  task_id: UUID;                    // FK → Task (required)
  title: string;                    // max 255 chars
  description: string | null;       // plain text or Markdown
  status: SubtaskStatus;
  progress: number;                 // 0–100, manual or auto on status change
  assignee_id: UUID | null;         // FK → People/HR
  start_date: Date | null;
  deadline: Date | null;
  sort_order: number;               // position within parent Task
  created_by: UUID;
  created_at: Timestamp;
  updated_at: Timestamp;
}

```

3.4 Comment

```typescript
interface Comment {
  id: UUID;
  entity_type: EntityType;          // 'story' | 'task' | 'subtask'
  entity_id: UUID;                  // FK → Story | Task | Subtask
  author_id: UUID;                  // FK → Users
  body: string;                     // rich text (Markdown)
  attachments: Attachment[];        // 0..N files
  is_edited: boolean;
  created_at: Timestamp;
  updated_at: Timestamp;
}

interface Attachment {
  id: UUID;
  comment_id: UUID;                 // FK → Comment
  file_name: string;
  file_size: number;                // bytes
  mime_type: string;
  storage_path: string;             // GCS path
  uploaded_by: UUID;
  uploaded_at: Timestamp;
}

```

3.5 Change Log

```typescript
interface ChangeLog {
  id: UUID;
  entity_type: EntityType;          // 'story' | 'task' | 'subtask'
  entity_id: UUID;
  action: ChangeAction;             // 'created' | 'updated' | 'deleted' | 'status_change
  field: string | null;             // e.g. 'status', 'priority', 'assignee_id'
  old_value: string | null;         // serialized previous value
  new_value: string | null;         // serialized new value
  changed_by: UUID;                 // FK → Users
  changed_at: Timestamp;
}

```

3.6 Enumerations

```typescript
type Priority = 'critical' | 'high' | 'medium' | 'low';

type StoryStatus = 'backlog' | 'active' | 'done' | 'closed';

type TaskStatus = 'backlog' | 'to_do' | 'in_progress' | 'in_review' | 'done';

type SubtaskStatus = 'to_do' | 'in_progress' | 'done';

type EntityType = 'story' | 'task' | 'subtask';

type ChangeAction = 'created' | 'updated' | 'deleted' | 'status_changed' | 'reassigned';

```

---

4. Status Flow & Automation

4.1 Status transitions per entity level

Story statuses: `Backlog` → `Active` → `Done` → `Closed`

Task statuses: `Backlog` → `To Do` → `In Progress` → `In Review` → `Done`

Subtask statuses: `To Do` → `In Progress` → `Done`

4.2 Status transition rules

All forward transitions are allowed (e.g. Backlog → In Progress is valid, skipping To Do). Backward transitions are also allowed (e.g. Done → In Progress for reopening). Status change always triggers a ChangeLog entry.

4.3 Cascade automation rules

Upward cascade (child → parent):

	Trigger	Effect
1	First Subtask moves to `In Progress`	Parent Task auto-moves to `In Progress` (if currently `Backlog` or `To Do`)
2	All Subtasks reach `Done`	Parent Task auto-moves to `Done` (with optional notification to assignee)
3	First Task moves to `In Progress` or beyond	Parent Story auto-moves to `Active` (if currently `Backlog`)
4	All Tasks reach `Done`	Parent Story auto-moves to `Done` (with optional notification to lead)

Downward cascade (parent → child): Not automatic. When a user manually changes a parent status, the system shows a confirmation dialog: "Do you want to update all child items to [status]?" — user can accept or decline.

4.4 Progress calculation

Subtask: Progress is set manually by the user OR auto-set based on status: `To Do` = 0%, `In Progress` = retains manual value (default 50% if never set), `Done` = 100%.

Task: Auto-calculated as the arithmetic mean of all child Subtask progress values. If a Task has no Subtasks, progress is set manually.

Story: Auto-calculated as the arithmetic mean of all child Task progress values. If a Story has no Tasks, progress is set manually.

Progress recalculates on every Subtask or Task status/progress change and propagates upward immediately.

---

5. CRUD Operations

5.1 Story CRUD

	Operation	Endpoint	Behaviour
	Create		

1		`POST /api/v1/stories`	Creates Story with `status: backlog`, `progress: 0`. Required: `title`. Optional: all other fields. Logs `ChangeLog(action: created)`
2	**Read**	`GET /api/v1/stories/:id`	Returns Story with aggregated `tasks_count`, `completed_tasks_count`, computed `progress`
3	**List**	`GET /api/v1/stories`	Paginated, filterable by status, priority, assignee, project, tags. Sortable by created_at, deadline, priority
4	**Update**	`PATCH /api/v1/stories/:id`	Partial update. Each changed field logged to ChangeLog. Status change triggers cascade check
5	**Delete**	`DELETE /api/v1/stories/:id`	Soft delete (sets `deleted_at`). Cascades to Tasks and Subtasks. Requires confirmation. Logs `ChangeLog(action: deleted)`

5.2 Task CRUD

	Operation	Endpoint	Behaviour
1	**Create**	`POST /api/v1/stories/:story_id/tasks`	Creates Task within Story. Required: `title`, `story_id`. `status: backlog`, `progress: 0`. Triggers Story progress recalculation
2	**Read**	`GET /api/v1/tasks/:id`	Returns Task with parent Story summary, child Subtasks list, computed progress
3	**List**	`GET /api/v1/stories/:story_id/tasks`	Tasks within a Story. Also: `GET /api/v1/tasks?assignee_id=X` for cross-story filtering
4	**Update**	`PATCH /api/v1/tasks/:id`	Partial update. Status change triggers upward cascade. Assignee change logs `reassigned` action
5	**Delete**	`DELETE /api/v1/tasks/:id`	Soft delete. Cascades to Subtasks. Triggers Story progress recalculation
6	**Reorder**	`PATCH /api/v1/tasks/:id/reorder`	Updates `sort_order`. Used for Kanban drag-and-drop within a column

5.3 Subtask CRUD

	Operation	Endpoint	Behaviour

1	**Create**	`POST /api/v1/tasks/:task_id/subtasks`	Creates Subtask within Task. Required: `title`, `task_id`. `status: to_do`, `progress: 0`. Triggers Task and Story progress recalculation
2	**Read**	`GET /api/v1/subtasks/:id`	Returns Subtask with parent Task and Story context
3	**List**	`GET /api/v1/tasks/:task_id/subtasks`	Subtasks within a Task
4	**Update**	`PATCH /api/v1/subtasks/:id`	Partial update. Status/progress change triggers upward cascade and progress recalculation
5	**Delete**	`DELETE /api/v1/subtasks/:id`	Hard delete (Subtasks are atomic). Triggers progress recalculation
6	**Reorder**	`PATCH /api/v1/subtasks/:id/reorder`	Updates `sort_order` within parent Task

---

6. Comments Flow

6.1 Behaviour

Comments can be attached to any entity (Story, Task, or Subtask). Each comment consists of a text body (Markdown) and optional file attachments.

6.2 Operations

	Operation	Endpoint	Behaviour
1	**Add comment**	`POST /api/v1/:entity_type/:entity_id/comments`	Creates comment. Required: `body`. Optional: `attachments[]` (multipart upload). Logs to ChangeLog
2	**List comments**	`GET /api/v1/:entity_type/:entity_id/comments`	Chronological order, paginated. Returns author name/avatar from Users
3	**Edit comment**	`PATCH /api/v1/comments/:id`	Author-only. Sets `is_edited: true`. Updates `updated_at`. Original text not preserved (not a versioned edit)

4	**Delete comment**	`DELETE /api/v1/comments/:id`	Author or admin. Cascades to attachments (deletes from storage)

6.3 Attachments

- Max file size: 25 MB per file
- Max files per comment: 10
- Allowed types: PDF, DOCX, XLSX, PNG, JPG, SVG, ZIP
- Storage: Google Cloud Storage, path pattern: `command/comments/{comment_id}/{file_name}`
- Files are not versioned; replacing requires deleting and re-uploading

---

7. Assignees

7.1 Integration with People/HR module

Assignee fields (`assignee_id` on Task/Subtask, `assigned_lead` on Story) reference the `personnel` table in the People/HR module. The Task Tracker does not maintain its own user directory.

7.2 Resolution

When displaying an assignee, the frontend fetches: `full_name`, `position`, `department`, `avatar_url` from the People/HR API. These are cached in Redis (TTL: 1 hour) to avoid repeated lookups.

7.3 Assignment rules

- A Task or Subtask can have 0 or 1 assignee (single assignee model)
- A Story has an `assigned_lead` — the person accountable for the Story's delivery
- Changing assignee triggers a `ChangeLog(action: reassigned)` entry
- Unassigned items are valid and appear in Backlog with an "Unassigned" indicator

---

8. Views

8.1 Backlog View (Table)

The default view for the Task Tracker. Displays all entities in a flat, filterable, sortable table with an expandable detail panel.

Table columns:

	Column	Description	Sortable	Filterable
1	**Type indicator**	Icon/badge: Story, Task, or Subtask	No	Yes (filter by type)
2	**Title**	Entity title with indentation by hierarchy level	Yes	Yes (text search)
	Status	Status badge with colour	Yes	Yes (multi-select)

3		coding		
4	**Priority**	Priority badge (Critical/High/Medium/Low)	Yes	Yes (multi-select)
5	**Assignee**	Avatar + name (or "Unassigned")	Yes	Yes (select person)
6	**Progress**	Progress bar (0–100%)	Yes	No
7	**Start date**	Date or "—"	Yes	Yes (date range)
8	**Deadline**	Date or "—". Red highlight if overdue	Yes	Yes (date range)
9	**Created**	Creation timestamp	Yes	No

Hierarchy display in table:

- Stories appear as top-level rows (no indentation)
- Tasks appear indented 1 level under their parent Story
- Subtasks appear indented 2 levels under their parent Task
- Each Story row has a collapse/expand toggle to show/hide child Tasks
- Each Task row has a collapse/expand toggle to show/hide child Subtasks
- Default state: Stories expanded, Tasks collapsed

Detail panel: Clicking any row opens a right-side detail panel (slide-over, not a new page) showing:

- Full title and description (editable inline)
- All metadata fields (status, priority, assignee, dates)
- Subtask list (for Tasks) or Task list (for Stories)
- Comments thread
- Change log (collapsible, chronological)
- Action buttons: Edit, Change Status, Delete

8.2 Kanban Board View

Displays Tasks as cards organised into columns by status. Stories serve as swim lanes or filters — not as cards on the board.

Columns (left to right):

`Backlog` → `To Do` → `In Progress` → `In Review` → `Done`

Card content:

- Task title
- Priority badge
- Assignee avatar
- Subtask progress indicator (e.g. "3/5 subtasks done")
- Deadline (with overdue indicator)
- Comment count icon

Interactions:

- **Drag and drop** cards between columns → updates Task status, triggers cascade automation, logs ChangeLog
- **Drag and drop** within a column → updates `sort_order`
- **Click card** → opens detail panel (same as Backlog view)
- **Quick-add** button at bottom of each column → creates new Task in that status
- **Expand card** → shows Subtask checklist inline on the card

Filters (top bar):

- Filter by Story (show only Tasks of selected Story)
- Filter by assignee
- Filter by priority
- Filter by deadline range

Swim lanes (optional toggle): When enabled, the board groups Tasks by parent Story. Each swim lane shows the Story title, progress bar, and priority. Tasks within each swim lane are arranged in their respective status columns.

---

9. Validation Rules

9.1 Field validations

	Field	Rule
1	`title`	Required. Min 3 chars, max 255 chars. No empty strings
2	`description`	Optional. Max 10,000 chars
3	`priority`	Required. Must be one of: `critical`, `high`, `medium`, `low`. Default: `medium`
4	`status`	Required. Must be valid for entity type. Default: `backlog` (Story/Task) or `to_do` (Subtask)
5	`assignee_id`	Optional. Must reference valid `personnel.id` in People/HR module
6	`start_date`	Optional. If set, must be ≤ `deadline`
7	`deadline`	Optional. If set, must be ≥ `start_date`
8	`progress`	0–100 integer. Auto-calculated for parent entities; manual for leaf entities
9	`estimated_hours`	Optional. Must be > 0 if set
10	`tags`	Optional. Max 10 tags, each max 50 chars

9.2 Business rule validations

- Cannot delete a Story that has Tasks with status `in_progress` or `in_review` without confirmation
- Cannot set a Task to `done` if it has Subtasks with status `to_do` or `in_progress` (prompt user to complete or skip)

- Deadline changes on a Story should warn if any child Task deadlines exceed the new Story deadline

---

10. Database Schema (PostgreSQL)

10.1 Tables

SQL

```sql
CREATE TABLE command_stories (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title           VARCHAR(255) NOT NULL,
    description     TEXT,
    status          VARCHAR(20) NOT NULL DEFAULT 'backlog',
    progress        SMALLINT NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),
    priority        VARCHAR(20) NOT NULL DEFAULT 'medium',
    project_id      UUID REFERENCES projects(id) ON DELETE SET NULL,
    tags            TEXT[] DEFAULT '{}',
    assigned_lead   UUID,
    start_date      DATE,
    deadline        DATE,
    sort_order      INTEGER NOT NULL DEFAULT 0,
    created_by      UUID NOT NULL REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at      TIMESTAMPTZ
);

CREATE TABLE command_tasks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    story_id        UUID NOT NULL REFERENCES command_stories(id) ON DELETE CASCADE,
    title           VARCHAR(255) NOT NULL,
    description     TEXT,
    status          VARCHAR(20) NOT NULL DEFAULT 'backlog',
    progress        SMALLINT NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),
    priority        VARCHAR(20) NOT NULL DEFAULT 'medium',
    assignee_id     UUID,
    start_date      DATE,
    deadline        DATE,
    estimated_hours NUMERIC(6,1),
    sort_order      INTEGER NOT NULL DEFAULT 0,
    created_by      UUID NOT NULL REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at      TIMESTAMPTZ
);

CREATE TABLE command_subtasks (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id         UUID NOT NULL REFERENCES command_tasks(id) ON DELETE CASCADE,
    title           VARCHAR(255) NOT NULL,
    description     TEXT,
    status          VARCHAR(20) NOT NULL DEFAULT 'to_do',
    progress        SMALLINT NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),
    assignee_id     UUID,
    start_date      DATE,
    deadline        DATE,
    sort_order      INTEGER NOT NULL DEFAULT 0,
    created_by      UUID NOT NULL REFERENCES users(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE command_comments (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type     VARCHAR(20) NOT NULL,
    entity_id       UUID NOT NULL,
    author_id       UUID NOT NULL REFERENCES users(id),
    body            TEXT NOT NULL,
    is_edited       BOOLEAN NOT NULL DEFAULT false,
```

```
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE command_attachments (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    comment_id      UUID NOT NULL REFERENCES command_comments(id) ON DELETE CASCADE,
    file_name       VARCHAR(255) NOT NULL,
    file_size       BIGINT NOT NULL,
    mime_type       VARCHAR(100) NOT NULL,
    storage_path    VARCHAR(500) NOT NULL,
    uploaded_by     UUID NOT NULL REFERENCES users(id),
    uploaded_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE command_changelog (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type     VARCHAR(20) NOT NULL,
    entity_id       UUID NOT NULL,
    action          VARCHAR(30) NOT NULL,
    field           VARCHAR(100),
    old_value       TEXT,
    new_value       TEXT,
    changed_by      UUID NOT NULL REFERENCES users(id),
    changed_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

10.2 Indexes

SQL

```sql
CREATE INDEX idx_tasks_story_id ON command_tasks(story_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_tasks_assignee ON command_tasks(assignee_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_tasks_status ON command_tasks(status) WHERE deleted_at IS NULL;
CREATE INDEX idx_subtasks_task_id ON command_subtasks(task_id);
CREATE INDEX idx_comments_entity ON command_comments(entity_type, entity_id);
CREATE INDEX idx_changelog_entity ON command_changelog(entity_type, entity_id);
CREATE INDEX idx_stories_project ON command_stories(project_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_stories_status ON command_stories(status) WHERE deleted_at IS NULL;

```

---

11. Integration Points

	System	Integration	Direction
1	**People/HR**	Resolve `assignee_id` → full name, position, avatar	Read
2	**Projects**	Link Story to project via `project_id`	Read
3	**Dashboard**	Feed task stats to dashboard widgets (via stats microservice pattern)	Write (events)
4	**v_critical_alerts**	Feed overdue tasks to the cross-module alert widget	Write (computed view)

11.1 Dashboard widget data

Following the established stats microservice pattern, create a `command_stats` table:

```sql
CREATE TABLE command_stats (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    snapshot_date           DATE NOT NULL,
    total_stories           INTEGER NOT NULL DEFAULT 0,
    active_stories          INTEGER NOT NULL DEFAULT 0,
    total_tasks             INTEGER NOT NULL DEFAULT 0,
    tasks_backlog           INTEGER NOT NULL DEFAULT 0,
    tasks_in_progress       INTEGER NOT NULL DEFAULT 0,
    tasks_done              INTEGER NOT NULL DEFAULT 0,
    overdue_tasks           INTEGER NOT NULL DEFAULT 0,
    avg_progress_pct        NUMERIC(5,2) NOT NULL DEFAULT 0,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Update trigger: Task/Story status change, cron fallback every 2 hours.

11.2 Alert items for critical alerts view

```sql
CREATE TABLE command_alert_items (
    id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type         VARCHAR(20) NOT NULL,
    entity_id           UUID NOT NULL,
    entity_title        VARCHAR(255) NOT NULL,
    assignee_id         UUID,
    assignee_name       VARCHAR(255),
    deadline            DATE,
    days_until_deadline INTEGER,
    alert_type          VARCHAR(50) NOT NULL,  -- 'overdue_task' | 'approaching_deadline'
    urgency_level       VARCHAR(20) NOT NULL,  -- 'critical' | 'warning'
    is_dismissed        BOOLEAN NOT NULL DEFAULT false,
    dismissed_by        UUID,
    dismissed_at        TIMESTAMPTZ,
    last_checked_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

These records feed into `v_critical_alerts` view following the existing platform pattern.

---

12. Non-functional Requirements

	Requirement	Target
1	API response time (list endpoints)	< 200ms for up to 500 items
2	Kanban drag-and-drop latency	< 100ms perceived (optimistic update)
3	File upload size	Max 25 MB per file
4	Concurrent users	Support 50 concurrent users
5	Data retention	Soft-deleted items retained for 90 days, then hard-deleted via cron

6	Change log retention	Indefinite (audit trail)
7	Browser support	Chrome 120+, Edge 120+, Safari 17+

---

13. Out of Scope (Phase 1)

The following features are planned for later phases and are explicitly excluded from Phase 1:

- Gantt / Timeline views (Phase 2)
- Task dependencies and critical path (Phase 2)
- Automated reports — morning digest, weekly, monthly (Phase 3)
- C-Level Board and KPI dashboard (Phase 4)
- Teams integration — call summary, task generation (Phase 5)
- WhatsApp/Telegram notifications (Phase 6)
- AI risk analyst (Phase 6)

This specification defines the technical scope for Phase 1. Implementation details (component structure, state management, API middleware) to be agreed with the development team during sprint planning.